Training Leaves Traces: Weight-Level Model Provenance Without Watermarks

Anonymous submission

Abstract

Model provenance depends on external evidence—hashes, metadata, logs, or watermarks—which can fail when checkpoints are adapted or records are incomplete. We ask whether provenance can be inferred from weights alone: if a suspect checkpoint descends from a reference model, does training leave a persistent structural signal? We find that modern residual architectures exhibit such a signal. In a two-layer residual branch with input projection W_{in} and output projection W_{out}, training couples paired matrices so that $W_{\text{out}}W_{\text{in}}$ exhibits a diagonal-dominance fingerprint: a signed, identity-like component absent from mismatched or random pairings. For deeper branches, we measure the trace on $M = W_K \cdots W_1$, formalize it with $s(M) = |\text{tr}(M)|/\|M\|_F$, derive a separation margin, and compute cross-block score matrices that recover residual-branch correspondences and yield checkpoint-level ancestry evidence.

Empirically, across X architectures, Y model families, Z post-training weight modifications, and N reference–suspect pairs, the fingerprint recovers X–Y% residual-block correspondences and achieves X AUROC / Y% TPR at Z% FPR for descendant detection. It is absent at initialization, appears after training, strengthens with width, and persists under fine-tuning, quantization, pruning, weight noise, and LoRA-style adaptation. When perturbations substantially degrade the fingerprint, model utility also drops by X%. Boundary cases clarify the signal: independently trained models do not falsely match, while distillation removes the trace despite functional imitation.

1 Introduction

The open-weight ecosystem has grown explosively: Hugging Face now hosts nearly one million model repositories, with derivatives proliferating faster than originals (Hugging Face 2024). When Meta's LLaMA weights leaked in March 2023, the community produced instruction-tuned variants—Alpaca, Vicuna, Koala—within weeks, each costing under $300 to create (Touvron et al. 2023). This rapid, low-cost adaptation means any widely-distributed checkpoint spawns hundreds of fine-tuned, merged, quantized, and renamed descendants whose lineage quickly becomes untraceable.

Current approaches to model provenance rely on externally maintained evidence. Model cards document training datasets and licensing terms (Mitchell et al. 2019); experiment tracking platforms such as MLflow and Weights & Biases record artifacts and lineage graphs; cryptographic hashes provide integrity checks for unmodified files. For cases where theft is anticipated, watermarking methods embed ownership signals into network parameters (Uchida et al. 2017) or decision boundaries (Adi et al. 2018) that can later be extracted to verify provenance.

Yet each approach has fundamental limitations. Cryptographic hashes become invalid after any weight modification—including routine quantization or fine-tuning. Metadata and provenance logs can be falsified, lost, or simply never recorded. Watermarks require deliberate insertion before deployment and remain vulnerable to removal: a recent systematization found that no surveyed DNN watermarking scheme proved robust against fine-tuning, pruning, or model extraction attacks (Lukas et al. 2022). These limitations leave a critical gap: there is no established method to verify provenance for models released without embedded watermarks or whose external records are unavailable.

Prior intrinsic fingerprinting methods have asked whether weight statistics—means, variances, or histogram features—can identify models (Zheng et al. 2022; Zhao et al. 2020), but these aggregate properties lack verification precision and are easily disrupted. Training dynamics research has established that gradient descent enforces near-orthogonal Jacobians in residual networks (Pennington, Schoenholz, and Ganguli 2017; Saxe, McClelland, and Ganguli 2014), yet this work focused on trainability, not on what structural traces the process leaves behind. We ask a different question: *does training itself imprint a recoverable signature—not in aggregate statistics, but in the relational structure between paired weight matrices?*

The answer is yes. In residual architectures, trained residual-branch products exhibit diagonal-dominant structure. This phenomenon emerges because residual blocks must maintain near-identity behavior for stable gradient flow—a requirement formalized as *dynamical isometry*, where the input-output Jacobian's singular values concentrate near unity throughout training (Pennington, Schoenholz, and Ganguli 2017; Saxe, McClelland, and Ganguli 2014). For a residual block computing $x + W_{\text{out}}\phi(W_{\text{in}}x)$, this constraint forces the branch product toward a negative scaled identity: $W_{\text{out}}W_{\text{in}} \approx -\varepsilon I$ (He et al. 2016). We find this fingerprint is absent at initialization, emerges

within the first five epochs of training, and yields a diagonal-dominance score that scales as $\Theta(\sqrt{d})$—separating correctly paired blocks from mismatched pairings by a margin that grows linearly with hidden dimension d.

This trace enables *passive* weight-level provenance verification: auditing whether a suspect checkpoint descends from a reference model without requiring watermarks, metadata, or cooperation from the original trainer. Unlike embedded watermarks that must be inserted before or during training, the diagonal-dominance fingerprint emerges from optimization itself and can be read retroactively from any residual model—including checkpoints deployed long before provenance verification was anticipated. The fingerprint maintains 100% pair accuracy across 21 attack configurations (fine-tuning, quantization, pruning, weight noise up to $\sim$20%), degrading only when perturbations also destroy model utility. Boundary cases clarify the signal's scope: independently trained models do not falsely match, while distillation removes the trace—correctly, since the method detects weight-level lineage rather than functional imitation.

We make the following contributions:

- **We formalize the diagonal-dominance signal and derive a separation margin that grows with width.** Building on Park (2026)'s observation that training induces negative diagonal structure in residual-branch products, we define the score $s(i,j) = |\mathrm{tr}(M)|/\|M\|_F$ and prove it achieves $\Theta(\sqrt{d})$ for correct pairs versus $\Theta(1/\sqrt{d})$ for mismatched pairs (Section 3).
- **We develop a passive provenance verification protocol.** Cross-block score matrices combined with Hungarian matching recover residual-branch correspondences and yield checkpoint-level ancestry evidence from weights alone—without watermarks, metadata, or cooperation from the original trainer (Section 4).
- **The fingerprint generalizes across architectures and survives post-training modification.** We validate across nine architecture families (GPT-2 to 1.5B, BERT, LLaMA, Mistral, Qwen, Gemma, ViT, Whisper, ResNet), demonstrate robustness under 21 attack configurations, and confirm via ablation that the signal requires both training and residual connections (Sections 5–6).

2 Background and Problem Setting

2.1 Passive Model Provenance

We define *passive model provenance* as follows: given a reference checkpoint R and a suspect checkpoint S with missing or untrusted metadata, determine whether S shares weight-level lineage with R after allowed post-training transformations. This is a *white-box* verification task—the verifier has access to both sets of weights. The setting matches regulatory audits, model hub compliance checks, and ownership disputes where both parties can produce their checkpoints.

The method detects *weight-level lineage*, not behavioral equivalence. Two models share weight-level lineage if the suspect's parameters can be derived from the reference through transformations such as fine-tuning, pruning, or quantization—without retraining from scratch. This is distinct from functional equivalence, where models produce similar outputs regardless of whether their weights share any relationship. Model extraction attacks (Tramèr et al. 2016) create functional copies with entirely fresh weights; such models are behaviorally similar but share no weight-level identity.

Distilled models and independently trained functional copies are explicitly outside the positive guarantee. A student trained via knowledge distillation (Hinton, Vinyals, and Dean 2015) may replicate the teacher's behavior with high fidelity, but it evolves its weights through a separate optimization process that produces distinct structural fingerprints. This boundary is intentional: proving that weights were copied or derived requires distinguishing "same functionality" from "same parameters (up to transformation)."

2.2 Limits of Existing Methods

Model provenance verification has relied on three families of approaches, each with fundamental limitations.

Cryptographic hashes fail immediately upon any weight modification. The avalanche effect ensures that fine-tuning, pruning, or even numerical precision changes produce entirely different digests, providing no signal of derivation.

Metadata and logging systems—model cards (Mitchell et al. 2019), experiment trackers like MLflow and Weights & Biases—require trusted provenance records that may be incomplete, falsified, or absent when models are redistributed outside controlled environments.

Watermarking methods embed ownership signals into weights (Uchida et al. 2017; Rouhani, Chen, and Koushanfar 2019) or train models to produce specific outputs on trigger inputs (Adi et al. 2018; Zhang et al. 2018). Both require forethought: the watermark must be inserted before training concludes, precluding retroactive verification of already-deployed models. A systematic evaluation found that no surveyed watermarking scheme is robust in practice against fine-tuning, pruning, and model extraction attacks (Lukas et al. 2022).

Output watermarking systems such as SynthID (Dathathri et al. 2024) identify AI-generated content by embedding statistical signatures during generation, but address a fundamentally different question: they detect whether content was AI-generated, not whether one model's weights derive from another's.

Decision-boundary fingerprinting (Cao, Jia, and Gong 2021; Le Merrer, Perez, and Trédan 2020) identifies functional similarity via adversarial examples near classification boundaries, but cannot distinguish weight lineage: a distilled student may share decision surfaces while having entirely independent weights.

The key differentiator of our approach: prior watermarking work asks how to *insert or protect* an ownership signal. We ask whether training already *leaves* a signal that can be read retroactively.

2.3 Residual Branch Products

Modern deep networks use *residual connections*—shortcuts that allow information to bypass layers—to enable stable

Architecture	Residual Form	Product M
BasicBlock / MLP	$x + W_2\phi(W_1x)$	W_2W_1
Transformer MLP	$x + W_2\phi(W_1x)$	W_2W_1
Attention V/O	$x + W_O\mathrm{Attn}(W_Vx)$	W_OW_V
Attention Q/K	$\mathrm{softmax}(QK^\top)$	$W_QW_K^\top$
Bottleneck	$x + W_3\phi(W_2\phi(W_1x))$	$W_3W_2W_1$
SwiGLU MLP	gated linear unit	$W_{\mathrm{d}}W_{\mathrm{u}}$

Table 1: Architecture-aware factorization for residual branch products.

training of very deep architectures (He et al. 2016). A residual block computes $x' = x + F(x)$, where x is the input and F is the residual branch function. For these blocks to maintain stable gradient flow, the Jacobian $J = I + J_F$ must remain close to orthogonal throughout training—a condition formalized as *dynamical isometry* (Pennington, Schoenholz, and Ganguli 2017; Saxe, McClelland, and Ganguli 2014).

For a residual branch with K linear transformations separated by nonlinearities, we define the *residual branch product*:

$$M = W_K W_{K-1} \cdots W_1 \in \mathbb{R}^{d\times d} \tag{1}$$

This product captures how the branch transforms the input space. The dynamical isometry constraint implies that M should approximate a negative scaled identity: $M \approx -\varepsilon I$ for small $\varepsilon > 0$ (Tarnowski et al. 2019; De and Smith 2020).

Table 1 shows the architecture-aware factorization for common residual blocks. Using the correct factorization is critical: the naive endpoint product W_3W_1 for Bottleneck ResNets (skipping the middle convolution) yields chance-level pairing accuracy, while the full triple product $W_3W_2W_1$ recovers 91–100% of correct pairs.

3 Training-Induced Diagonal Dominance

3.1 Empirical Observation

Training a deep residual network imprints a characteristic structure on its weights. We first demonstrate this phenomenon empirically before formalizing it.

At initialization, no diagonal structure appears. We compute the score matrix $s(i, j)$ for all pairs of input and output projections in a 48-block residual network. At epoch 0, the matrix shows no discernible pattern: Hungarian matching recovers only 6.25% of correct pairs (chance is $1/48 \approx 2.1\%$), and pair separation—the gap between the worst correct-pair score and the best incorrect-pair score—is -0.42 (negative indicates no separation). Across seven initialization schemes—Kaiming uniform/normal, Xavier uniform/normal, Gaussian, uniform, and orthogonal—pair accuracy remains at chance level ($\leq 2.1\%$). Critically, orthogonal initialization provides dynamical isometry at initialization by construction (Saxe, McClelland, and Ganguli 2014), yet shows *zero* correct pairs before training, ruling out the hypothesis that near-orthogonal Jacobians alone create the fingerprint.

During training, diagonal structure emerges rapidly. By epoch 5, the diagonal of $s(i, j)$ is fully separated and Hungarian matching recovers 100% of correct pairs. The mean correct-pair score rises monotonically from 0.19 at epoch 0 to 2.07 at epoch 300, while the mean incorrect-pair score remains flat at $\approx$0.16—a $21\times$ ratio at convergence. The transformation occurs within the first few epochs, well before training loss plateaus.

The effect is not explained by matrix shape or architecture alone. A null model with randomly initialized weights achieves only 6.25% pair accuracy with negative separation, ruling out shape compatibility as an explanation. To confirm the fingerprint requires residual training specifically, we train a PlainNet—architecturally identical except that skip connections are removed. While both networks converge to similar evaluation loss (1.35 vs 1.56), ResNet achieves 100% pair accuracy (AUC 0.98) with 100% of correctly paired products exhibiting negative trace, while PlainNet achieves only 3%—at the chance baseline of $1/24 \approx 4.2\%$—with AUC 0.51 and no discernible diagonal structure. The skip connection creates a functional constraint—the block must act as a near-identity perturbation to preserve gradient flow—that imprints the characteristic trace structure.

3.2 Diagonal-Dominance Fingerprint

For a candidate residual branch product $M_{ij} = W_{\mathrm{out}}^{(j)} W_{\mathrm{in}}^{(i)}$, we define the *diagonal-dominance score*:

$$s(i, j) = \frac{|\mathrm{tr}(M_{ij})|}{\|M_{ij}\|_F} \tag{2}$$

The numerator $|\mathrm{tr}(M)|$ captures *diagonal mass*—how much of the matrix energy concentrates on the diagonal. The denominator $\|M\|_F$ normalizes by total matrix energy (Frobenius norm), isolating structural fingerprint from scale.

This ratio has a natural geometric interpretation. A matrix with energy uniformly spread across entries achieves $s = \Theta(1/\sqrt{d})$, while a matrix proportional to the identity achieves $s = \sqrt{d}$. The key insight is that correctly paired blocks—where all K matrices come from the same residual branch—score at $\Theta(\sqrt{d})$, while incorrect pairings remain at the random baseline $\Theta(1/\sqrt{d})$. This yields a signal-to-noise gap that grows linearly with hidden dimension d, making correct pairs increasingly separable in wider networks.

Dynamical isometry (Pennington, Schoenholz, and Ganguli 2017) predicts that a well-trained residual block has Jacobian $J = I + W_{\mathrm{out}}W_{\mathrm{in}}$ close to orthogonal. For the block to act as a near-identity perturbation of the residual stream, the branch product must approximate a negative scaled identity. We formalize this with the decomposition:

$$M = W_{\mathrm{out}}^{(i)} W_{\mathrm{in}}^{(i)} = -\varepsilon I + E \tag{3}$$

where $\varepsilon = |\mathrm{tr}(M)|/d > 0$ captures diagonal strength and E is the residual with $\mathrm{tr}(E) = 0$. The matrix E contains all off-diagonal structure plus the zero-trace portion of the diagonal. Dynamical isometry implies $\|E\|_F \ll \varepsilon\sqrt{d}$ for well-trained blocks; the following proposition quantifies the margin exactly.

Crucially, diagonal dominance captures *relational* structure between paired matrices—how they jointly encode the

near-identity mapping—rather than aggregate properties of individual matrices. This relational fingerprint is preserved under fine-tuning and noise because it reflects the functional constraint that the residual block must approximate.

3.3 Margin Analysis and Null Model

Proposition 1 (Margin Formula)**.** *Let $M = -\varepsilon I + E$ with $\varepsilon = |\mathrm{tr}(M)|/d$ and $\mathrm{tr}(E) = 0$. Then the diagonal-dominance score satisfies*

$$s(i,i) = \frac{\sqrt{d}}{\sqrt{1 + \|E\|_F^2/(\varepsilon^2 d)}} \leq \sqrt{d}, \tag{4}$$

with equality if and only if $E = 0$, i.e., $M = -\varepsilon I$.

Proof. By construction, $|\mathrm{tr}(M)| = \varepsilon d$. For the Frobenius norm, expand $\|M\|_F^2 = \|-\varepsilon I + E\|_F^2 = \varepsilon^2 d - 2\varepsilon\,\mathrm{tr}(E) + \|E\|_F^2 = \varepsilon^2 d + \|E\|_F^2$, where the last step uses $\mathrm{tr}(E) = 0$. Substituting into (2): $s(i,i) = \varepsilon d/\sqrt{\varepsilon^2 d + \|E\|_F^2} = \sqrt{d}/\sqrt{1 + \|E\|_F^2/(\varepsilon^2 d)}$. The bound follows immediately, with equality iff $\|E\|_F = 0$. □

The margin formula reveals the geometry of diagonal dominance. The numerator $\sqrt{d}$ is the score achievable by a perfect negative-scalar identity; the denominator penalizes deviation from this ideal via the dimensionless ratio $\|E\|_F^2/(\varepsilon^2 d)$. For typical trained blocks, $\|E\|_F/(\varepsilon\sqrt{d}) \in [1.9, 3.8]$, yielding $s(i,i) \in [1.76, 3.23]$—well above the baseline but below the theoretical maximum of $\sqrt{d}$.

Corollary 1 (Signal-to-Baseline Ratio)**.** *Suppose $(W_{\text{in}}^{(i)}, W_{\text{out}}^{(j)})$ with $i \neq j$ are independent random matrices with i.i.d. zero-mean entries. Then $\mathbb{E}[s(i,j)^2] = 1/d$, so $\mathbb{E}[s(i,j)] \approx 1/\sqrt{d}$. The signal-to-baseline ratio scales as d.*

Correct pairs achieve scores of order $\sqrt{d}$, while incorrect pairs score at the random baseline $1/\sqrt{d}$. The gap grows with dimension, providing increasingly robust separation for wider networks. On GPT-2 models, mean $s(i,i)$ rises from 4.18 ($d = 768$, 124M parameters) to 7.77 ($d = 1600$, 1.5B parameters), consistent with $\sqrt{d}$ scaling.

Corollary 2 (Null Model)**.** *For randomly initialized weights with no training, all pairs satisfy $\mathbb{E}[s(i,j)] = 1/\sqrt{d} + o(1)$ uniformly in (i,j). Hungarian matching recovers correct pairs at the chance rate $1/L$.*

The null model rules out three alternative explanations: (a) diagonal dominance is not an artifact of compatible matrix shapes; (b) the residual connection itself does not create the signal; (c) any nontrivial loss level does not suffice. The diagonal-dominance fingerprint is a *learned* property induced by gradient descent. The theory explains why the observed structure is identifiable once induced; controlled experiments—including the initialization ablation showing orthogonal init at 0% pairing and the PlainNet control at 3%—confirm that training induces it

4 From Fingerprints to Provenance Verification

This section develops the computational machinery for transforming the diagonal-dominance fingerprint into a practical provenance verification system. We present: (1) an algorithm for recovering residual-block correspondences between checkpoints, (2) a model-level lineage score with statistical calibration, and (3) an end-to-end verification protocol suitable for deployment.

4.1 Pairing Algorithm

Given a reference model R with L residual blocks and a suspect model S with L blocks, we seek to determine which block in S corresponds to which block in R. Let $\{(W_{\text{in}}^{(i)}, W_{\text{out}}^{(i)})\}_{i=1}^{L}$ denote the input and output projections of R's residual branches, and $\{(\tilde{W}_{\text{in}}^{(j)}, \tilde{W}_{\text{out}}^{(j)})\}_{j=1}^{L}$ denote those of S. If S descends from R—through fine-tuning, quantization, or other post-training modifications—then a permutation $\pi : [L] \to [L]$ should exist such that block j in S corresponds to block $\pi(j)$ in R.

Score Matrix Construction. We construct a score matrix $\mathbf{S} \in \mathbb{R}^{L \times L}$ where each entry quantifies the diagonal-dominance evidence that block i in the reference corresponds to block j in the suspect:

$$s(i,j) = \frac{|\mathrm{tr}(M_{ij})|}{\|M_{ij}\|_F}, \quad \text{where } M_{ij} = \tilde{W}_{\text{out}}^{(j)} W_{\text{in}}^{(i)}. \tag{5}$$

The cross-model product M_{ij} tests whether the output projection from suspect block j and the input projection from reference block i exhibit the trained coupling characteristic of a true residual branch. If S descends from R and block j corresponds to block i, then M_{ij} inherits the diagonal-dominant structure of the original branch product, yielding $s(i,j) = \Theta(\sqrt{d})$. For non-corresponding pairs, the matrices lack this coupling, and $s(i,j)$ remains at the random baseline $\Theta(1/\sqrt{d})$.

Optimal Assignment. Given the score matrix $\mathbf{S}$, we seek a one-to-one assignment $\pi : [L] \to [L]$ that maximizes total evidence for correct correspondences. This is the classical optimal assignment problem, solvable via the Hungarian algorithm (Kuhn 1955):

$$\pi^* = \arg\max_{\pi \in \mathcal{P}_L} \sum_{i=1}^{L} s(i, \pi(i)), \tag{6}$$

where $\mathcal{P}_L$ denotes the set of all permutations on $[L]$. For derived models with preserved block ordering, the optimal assignment recovers the identity permutation: $\pi^*(i) = i$ for all $i \in [L]$.

Complexity. Score matrix computation requires $O(L^2 d^2 h)$ operations, where h is the intermediate dimension (typically $4d$ for transformer MLPs). The Hungarian algorithm runs in $O(L^3)$. For GPT-2-XL ($L = 48$, $d = 1600$), verification completes in approximately 10 seconds on a single GPU.

Empirical Validation. On GPT-2 ($L = 12$, $d = 768$), the pairing algorithm achieves 100% pair accuracy with mean

diagonal score $\bar{s}(i,i) = 4.18$ and mean off-diagonal score $\bar{s}(i,j) = 0.12$ for $i \neq j$—a $35\times$ separation ratio. At random initialization, pair accuracy drops to 6.25% (chance level). The algorithm relates to permutation-based alignment methods for model merging (Ainsworth, Hayase, and Srinivasa 2023) and federated learning (Singh and Jaggi 2020), but differs in objective: we match blocks to establish lineage evidence rather than to enable weight interpolation.

4.2 Model-Level Lineage Score

Block-level pairing establishes correspondences, but provenance decisions require a scalar statistic. We aggregate block-level similarity into a model-level lineage score with statistical calibration.

Centered Diagonal Fingerprint. For each branch ℓ, we compute the centered diagonal fingerprint that isolates checkpoint-specific structure:

$$\psi_\ell(M) = \frac{\mathrm{diag}(M_\ell) - \bar{d}_\ell}{\|\mathrm{diag}(M_\ell) - \bar{d}_\ell\|_2}, \tag{7}$$

where $M_\ell = W_{\text{out}}^{(\ell)} W_{\text{in}}^{(\ell)}$ is the branch product and $\bar{d}_\ell$ is the mean diagonal entry. Centering removes the generic diagonal-dominant component shared across trained residual models.

Lineage Score. The model-level lineage score between checkpoints A and B is the mean cosine similarity of their aligned centered diagonal fingerprints:

$$\mathcal{L}(A,B) = \frac{1}{L}\sum_{\ell=1}^{L} \langle \psi_\ell^A, \psi_\ell^B \rangle. \tag{8}$$

This score satisfies: (1) self-similarity $\mathcal{L}(A,A) = 1$; (2) symmetry $\mathcal{L}(A,B) = \mathcal{L}(B,A)$; (3) boundedness $\mathcal{L}(A,B) \in [-1,1]$; and (4) orthogonality $\mathbb{E}[\mathcal{L}(A,B)] \approx 0$ when A and B are independently trained.

Statistical Calibration. We calibrate against a null distribution of unrelated models. Let $\mathcal{N}_A = \{\mathcal{L}(A,U_j) : U_j \notin \mathcal{D}(A)\}_{j=1}^{n}$ be scores between reference A and independently trained models U_j. The calibrated z-score is:

$$Z(A,B) = \frac{\mathcal{L}(A,B) - \mu_{\text{null}}}{\sigma_{\text{null}}}, \tag{9}$$

where μ_{null} and σ_{null} are the mean and standard deviation of $\mathcal{N}_A$.

Empirical Score Distributions. Table 2 reports lineage scores across suspect categories.

The separation is striking: descendants achieve $\mathcal{L} > 0.98$ while non-descendants remain below $\mathcal{L} < 0.03$, yielding a margin of 0.96—over 96% of the full score range. Descendants produce z-scores of $Z \approx 83$ (mean), while non-descendants remain at $Z \approx 0$ by construction. This extreme separation—over 80 standard deviations—enables perfect classification (AUROC $= 1.0$, TPR@1%FPR $= 100\%$).

Lineage Chain Decay. For checkpoints along a fine-tuning chain $C_1 \to C_2 \to \cdots \to C_5$, the lineage score decays monotonically: $\mathcal{L}(C_1,C_2) = 0.9997$, $\mathcal{L}(C_1,C_3) = 0.9994$, $\mathcal{L}(C_1,C_5) = 0.9986$. This provides genealogical ordering: $\mathcal{L}(A,B) > \mathcal{L}(A,C)$ implies B is genealogically closer to A than C.

Suspect Type	Mean $\mathcal{L}$	Min	Max
Fine-tuned	0.998	0.991	1.000
Quantized (8-bit)	0.999	0.995	1.000
Pruned (30%)	0.997	0.989	1.000
LoRA-merged	0.996	0.985	1.000
Independent	0.003	-0.021	0.028
Random init	0.001	-0.018	0.019
Distilled	0.002	-0.015	0.022

Table 2: Lineage score distributions by suspect category. Descendants cluster near $\mathcal{L} = 1$; non-descendants near $\mathcal{L} = 0$.

Transformation	Parameters	Detection
Fine-tuning	LR 10^{-5}–10^{-3}	100%
Quantization	16-bit, 8-bit	100%
Quantization	6-bit	94%
Gaussian noise	$\sigma \leq 0.3$	100%
Gaussian noise	$\sigma = 0.5$	96%
Pruning	up to 30%	100%
LoRA merge	rank 8–64	100%

Table 3: Verification robustness. Detection rate is fraction of true descendants correctly identified at $\tau = 1.645$.

4.3 Verification Protocol

We formalize the end-to-end verification procedure. The protocol takes a reference checkpoint A, suspect checkpoint B, and pre-calibrated null distribution $\mathcal{N}$, outputting one of four verdicts.

Protocol Steps:

1. **Compatibility Check:** Verify $\mathrm{arch}(A) = \mathrm{arch}(B)$. If not, return INCOMPATIBLE.
2. **Extract:** Compute architecture-aware branch products $\{M_\ell^A\}$, $\{M_\ell^B\}$ using Table 1.
3. **Fingerprint:** Compute centered diagonal fingerprints $\{\psi_\ell^A\}$, $\{\psi_\ell^B\}$.
4. **Align:** If block correspondence is unknown, apply Hungarian matching (Section 4.1).
5. **Aggregate:** Compute lineage score $\mathcal{L}(A,B)$ via Eq. (8).
6. **Calibrate:** Compute z-score $Z(A,B)$ via Eq. (9).
7. **Decide:** Return verdict based on thresholds $\tau_{\text{upper}}, \tau_{\text{lower}}$:

$$\text{Verdict} = \begin{cases} \textsc{Descendant} & \text{if } Z(A,B) > \tau_{\text{upper}} \\ \textsc{Non-Descendant} & \text{if } Z(A,B) < \tau_{\text{lower}} \\ \textsc{Inconclusive} & \text{otherwise} \end{cases} \tag{10}$$

Threshold Selection. Under the Gaussian approximation for the null distribution, $\tau = 1.645$ yields 95% confidence; $\tau = 2.576$ yields 99% confidence. For high-stakes applications (legal disputes, regulatory audits), we recommend $\tau_{\text{upper}} = 3.0$, yielding theoretical FPR $< 0.13\%$.

Robustness Guarantees. Table 3 summarizes detection rates under transformations.

Failure Modes. The protocol cannot provide a verdict when: (1) architectures differ (returns INCOMPATIBLE); (2) perturbation is extreme enough to destroy utility (e.g., 2-bit

quantization); (3) both models descend from a common ancestor not available for comparison; (4) adversarial evasion specifically targets the fingerprint (analyzed in Section 6).

Comparison with Watermarking. Unlike watermarking (Uchida et al. 2017; Adi et al. 2018), our protocol requires no insertion during training, enables retroactive verification, and survives fine-tuning. The key distinction: watermarks embed artificial signals removable without affecting function, while diagonal dominance arises from optimization itself and cannot be removed without degrading model utility.

5 Experiments

Our experiments are organized around four questions that validate the core provenance claim in increasing order of difficulty: the fingerprint must first exist, then generalize, then survive realistic checkpoint modifications, and finally discriminate true weight-level descendants from non-descendants. We defer adaptive evasion, reparameterization attacks, and boundary cases such as distillation and shared ancestry to Section 6.

Throughout this section, we evaluate the diagonal-dominance score

$$s(M) = \frac{|\operatorname{tr}(M)|}{\|M\|_F}, \tag{11}$$

where M is an architecture-aware residual branch product. For model-level verification, we use the centered diagonal fingerprint and lineage score defined in Section 4. Unless otherwise stated, reported results include multiple random seeds and confidence intervals.

5.1 Exists: Does Residual Training Induce a Diagonal-Dominance Fingerprint?

Purpose. This experiment tests the core scientific claim: residual training induces a measurable diagonal-dominance fingerprint in weight space. The goal is to rule out simpler explanations, including initialization artifacts, compatible matrix shapes, residual architecture alone, or the scoring function itself.

Experiment. We train residual networks from scratch and measure diagonal-dominance throughout training. For each checkpoint, we compute the score matrix $S \in \mathbb{R}^{L\times L}$, where

$$S_{ij} = \frac{|\operatorname{tr}(W_{\text{out}}^{(j)} W_{\text{in}}^{(i)})|}{\|W_{\text{out}}^{(j)} W_{\text{in}}^{(i)}\|_F}. \tag{12}$$

Correct residual-branch pairings correspond to the diagonal entries S_{ii}; mismatched pairings correspond to off-diagonal entries S_{ij}, $i \neq j$.

We compare checkpoints at initialization, early training, mid-training, and convergence. To isolate the role of residual training, we include a PlainNet control with the same layer shapes but without skip connections. We also evaluate multiple initialization schemes, including Kaiming, Xavier, Gaussian, uniform, and orthogonal initialization.

Metrics. We report: mean correct-pair score $\mathbb{E}[S_{ii}]$; mean incorrect-pair score $\mathbb{E}[S_{ij}]$, $i \neq j$; correct–incorrect score gap; correct/incorrect AUROC; Hungarian matching pair accuracy; trace-sign consistency across correctly paired products; first epoch at which correct pairs separate from mismatched pairs; and confidence intervals across random seeds.

Baselines. We compare against: randomly initialized residual networks; orthogonally initialized residual networks; shuffled residual-branch pairings; randomly generated same-shape matrix products; PlainNet models without residual connections; and individual weight statistics such as Frobenius norm, mean, variance, and histogram features.

Exit Criteria. This question is answered positively if: (1) the fingerprint is absent or near chance at initialization; (2) correct residual-branch products separate from mismatched products after training; (3) the effect is consistent across seeds; and (4) the PlainNet and shuffled-pair controls fail to produce comparable separation.

5.2 Generalizes: Does the Fingerprint Transfer Across Architectures?

Purpose. This experiment tests whether the fingerprint is a general property of trained residual branches or a narrow artifact of a single architecture. It also evaluates whether architecture-aware factorization is necessary for extracting the signal.

Experiment. We evaluate the fingerprint across multiple residual model families and branch types. Candidate architectures include convolutional residual networks, transformer language models, vision transformers, and gated-MLP transformer variants. For each architecture, we compute the residual branch product M using the factorization appropriate to that block.

For a two-layer residual MLP block, $M = W_2W_1$. For a bottleneck residual block, $M = W_3W_2W_1$. For attention value-output branches, $M = W_OW_V$. For gated MLPs, we evaluate the architecture-specific product implied by the effective residual branch. We compare correct factorizations against incorrect or incomplete products, such as endpoint-only products that skip intermediate projections.

Metrics. We report: pair accuracy by architecture; correct/incorrect AUROC by architecture; mean correct-pair and incorrect-pair scores; score separation ratio; scaling of the score with hidden dimension; sensitivity to depth and branch type; and runtime and memory cost of fingerprint extraction.

Baselines. We compare against: incorrect branch factorizations; endpoint-only products; random pairings with matched shapes; individual layer statistics without branch products; full-layer cosine similarity; singular value spectrum similarity; and weight histogram similarity.

Exit Criteria. This question is answered positively if the fingerprint appears across multiple residual architectures when the correct branch product is used, and if incorrect or incomplete factorizations substantially reduce the signal.

5.3 Survives: Does the Fingerprint Persist Under Post-Training Transformations?

Purpose. This experiment tests whether the fingerprint remains detectable after common checkpoint modifications. This is necessary for practical provenance verification, because real model descendants are often fine-tuned, quantized, pruned, adapted, or otherwise modified before redistribution.

Experiment. Starting from a reference checkpoint A, we construct transformed descendant checkpoints B using common post-training transformations: continued fine-tuning; instruction tuning; LoRA training followed by adapter merge; quantization at multiple bit widths; unstructured and structured pruning; additive weight noise; adapter insertion and removal; and model merging or model soups where applicable.

For each transformed checkpoint B, we compute the block-level pairing scores and the model-level lineage score $\mathcal{L}(A, B)$. We also evaluate downstream model utility to determine whether fingerprint degradation coincides with utility degradation.

Metrics. We report: lineage score $\mathcal{L}(A, B)$; calibrated lineage z-score; detection rate at a fixed threshold; TPR at fixed FPR; AUROC across transformation strengths; Hungarian matching pair accuracy after transformation; downstream utility before and after transformation; utility drop versus lineage-score drop; and transformation strength at which detection fails.

Baselines. We compare against: cryptographic hashes; full-model weight cosine similarity; full-model ℓ_2 distance; layerwise cosine similarity; weight histogram similarity; singular value spectrum similarity; random unrelated models with the same architecture; and same-architecture models trained independently from scratch.

Exit Criteria. This question is answered positively if transformed descendants remain above the lineage threshold under common utility-preserving transformations. The fingerprint may degrade as transformation strength increases, but degradation should be gradual and should coincide with measurable utility loss.

5.4 Discriminates: Can the Fingerprint Distinguish Descendants from Non-Descendants?

Purpose. This experiment tests the central provenance claim: the fingerprint should distinguish true weight-level descendants from unrelated or merely functionally similar models.

Experiment. We construct a labeled benchmark of reference–suspect checkpoint pairs. Positive pairs share weight-level lineage: fine-tuned descendants; quantized descendants; pruned descendants; LoRA-merged descendants; lightly noised descendants; and multi-step fine-tuning chains.

Negative pairs do not share weight-level lineage: independently trained models with the same architecture; same-architecture models trained on the same data with different seeds; same-architecture models trained on different data; randomly initialized models; distilled students; functionally similar models without shared weights; and unrelated model families.

For each pair (A, B), we compute the aligned centered diagonal fingerprints and the model-level lineage score $\mathcal{L}(A, B)$. We calibrate thresholds using an empirical null distribution of unrelated checkpoints.

Metrics. We report: AUROC; AUPRC; TPR at 1% FPR; TPR at 0.1% FPR; FPR at the selected operating threshold; precision under realistic low-base-rate settings; empirical null quantiles; calibrated z-score distributions; and lineage score distributions by suspect category.

For fine-tuning chains $A \to B \to C$, we additionally test genealogical ordering: whether $\mathcal{L}(A, B) > \mathcal{L}(A, C)$, indicating that the lineage score reflects relative proximity to the reference checkpoint.

Baselines. We compare against: raw weight cosine similarity; layerwise weight cosine similarity; full-model ℓ_2 distance; centered kernel alignment or representation similarity where activations are available; output-distribution similarity; behavioral benchmark similarity; decision-boundary fingerprinting where feasible; and weight histogram and statistical fingerprinting baselines.

Exit Criteria. This question is answered positively if true descendants score consistently above the calibrated threshold while non-descendants remain near the null distribution. Independently trained same-architecture models and distilled models should not falsely appear as descendants.

6 Adaptive Evasion and Boundary Cases

6.1 Direct Fingerprint-Suppression Attacks

Purpose: Reviewer-proof the paper against the obvious attack.

Content: Attack objective:

$$\min_{\Delta W} L(A, A + \Delta W) \tag{13}$$

subject to preserving model utility.

Evaluate whether an adversary can reduce the lineage score while keeping the model useful.

6.2 Function-Preserving Reparameterizations

Purpose: Address the basis-dependence objection.

Experiments / analysis: Consider:

- Orthogonal rotations
- Neuron/channel permutations
- Invertible basis changes where architecture permits
- Equivalent reparameterizations across adjacent layers
- Adapter insertion/removal

6.3 Distillation and Retraining

Purpose: Define the boundary cleanly.

Content: State:

> The fingerprint tracks weight-level lineage, not function-level imitation.

Evaluate:

- Distilled students
- Retrained-from-scratch models
- Independently trained models with same architecture
- Same architecture/same data/different seed

7 Related Work

Purpose: Situate the contribution without bloating the paper.

Recommended subsections:

1. Neural Network Watermarking

- Embedded parameter watermarks
- Backdoor/trigger watermarks
- Watermark capacity
- NeuralMark / robust weight-based watermarking

2. Model Fingerprinting and Ownership Verification

- Black-box fingerprints
- Decision-boundary fingerprints
- Intrinsic/statistical fingerprints

3. Training Dynamics and Residual Networks

- Dynamical isometry
- Residual near-identity behavior
- Weight-space structure

4. Model Provenance and Trustworthy ML

- Model supply chains
- Checkpoint auditability
- Lineage verification

8 Discussion and Limitations

Purpose: Show maturity and avoid overclaiming.

Must say clearly:

- White-box only
- Residual architectures only
- Detects weight-level lineage, not behavioral identity
- Distillation/retraining breaks or erases the signal
- Not a proof of safety
- Not a replacement for watermarking
- Adaptive reparameterization attacks require care
- Provenance is one part of an audit stack

9 Conclusion

Purpose: End with the scientific thesis.

Recommended message:

> Training itself can provide provenance signal. Residual-network optimization leaves an intrinsic weight-space trace; diagonal dominance exposes it; and this trace enables passive model-lineage auditing under common post-training transformations.

References

Adi, Y.; Baum, C.; Cisse, M.; Pinkas, B.; and Keshet, J. 2018. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX Security Symposium*, 1615–1631.

Ainsworth, S. K.; Hayase, J.; and Srinivasa, S. 2023. Git Re-Basin: Merging Models modulo Permutation Symmetries. In *International Conference on Learning Representations*.

Cao, X.; Jia, J.; and Gong, N. Z. 2021. IPGuard: Protecting Intellectual Property of Deep Neural Networks via Fingerprinting the Classification Boundary. In *AsiaCCS*, 14–25.

Dathathri, S.; See, A.; Ghaisas, S.; Huang, P.-S.; McAdam, R.; Welbl, J.; Bachani, V.; Kaskasoli, A.; Sherborne, T.; et al. 2024. Scalable Watermarking for Identifying Large Language Model Outputs. *Nature*.

De, S.; and Smith, S. L. 2020. Batch Normalization Biases Residual Blocks Towards the Identity Function in Deep Networks. In *NeurIPS*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Hugging Face. 2024. Hugging Face Model Hub. https://huggingface.co/models. Accessed: 2024.

Kuhn, H. W. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1-2): 83–97.

Le Merrer, E.; Perez, P.; and Trédan, G. 2020. Adversarial Frontier Stitching for Remote Neural Network Watermarking. *Neural Computing and Applications*, 32(13): 9233–9244.

Lukas, N.; Jiang, E.; Li, X.; and Kerschbaum, F. 2022. SoK: How robust is image classification deep neural network watermarking? In *2022 IEEE Symposium on Security and Privacy (SP)*, 787–804. IEEE.

Mitchell, M.; Wu, S.; Zaldivar, A.; Barnes, P.; Vasserman, L.; Hutchinson, B.; Spitzer, E.; Raji, I. D.; and Gebru, T. 2019. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–229.

Park, H. 2026. I Dropped a Neural Net. *arXiv preprint arXiv:2602.19845*.

Pennington, J.; Schoenholz, S.; and Ganguli, S. 2017. Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice. In *Advances in Neural Information Processing Systems*, volume 30.

Rouhani, B. D.; Chen, H.; and Koushanfar, F. 2019. DeepSigns: An End-to-End Watermarking Framework for Deep Neural Networks. In *ASPLOS*, 485–497.

Saxe, A. M.; McClelland, J. L.; and Ganguli, S. 2014. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations*.

Singh, S. P.; and Jaggi, M. 2020. Model Fusion via Optimal Transport. In *Advances in Neural Information Processing Systems*, volume 33, 22045–22055.

Tarnowski, W.; Warchol, P.; Jastrzebski, S.; Tabor, J.; and Nowak, M. 2019. Dynamical Isometry is Achieved in Residual Networks in a Universal Way for Any Activation Function. In *AISTATS*, 2221–2230.

Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Tramèr, F.; Zhang, F.; Juels, A.; Reiter, M. K.; and Ristenpart, T. 2016. Stealing Machine Learning Models via Prediction APIs. In *USENIX Security Symposium*, 601–618.

Uchida, Y.; Nagai, Y.; Sakazawa, S.; and Satoh, S. 2017. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*, 269–277.

Zhang, J.; Gu, Z.; Jang, J.; Wu, H.; Stoecklin, M. P.; Huang, H.; and Molloy, I. 2018. Protecting Intellectual Property of Deep Neural Networks with Watermarking. In *ASIACCS*, 159–172.

Zhao, W.; Rao, S.; Li, J.; and Xiong, Z. 2020. Shaping deep feature space towards gaussian mixture for visual classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 3255–3264.

Zheng, Z.; Zhang, H.; Chen, J.; and Li, B. 2022. Fingerprinting deep neural networks globally via universal adversarial perturbations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 13430–13439.